

Laboratorio de productores y consumidores

El objetivo es que el consumidor reciba cada dato del productor una sola vez, sin perder ni repetir valores.

a Compilación y prueba inicial

¿Funciona correctamente sin sincronización? ¿Qué problemas pueden aparecer?

No siempre. El productor puede reemplazar un dato antes de que se consuma, y el consumidor puede leer cuando el recipiente está vacío. Que funcione una vez no garantiza que siempre lo haga.

¿Por qué pueden cambiar los resultados entre ejecuciones?

Los hilos no avanzan al mismo ritmo. Las pausas aleatorias cambian quién llega primero al recipiente, por eso una versión sin sincronización puede dar resultados distintos.

¿Qué mecanismos hacen falta para garantizar un funcionamiento correcto?

Ambos hilos deben usar el mismo recipiente. Se necesita `synchronized` para protegerlo, `wait()` para esperar y `notifyAll()` para avisar al otro hilo.

b Implementación de la sincronización

¿Cómo se coordinan produce y consume?

El productor espera si el recipiente está lleno y el consumidor espera si está vacío. La variable `producido` indica ese estado. Después de producir o consumir, se cambia la variable y se avisa al otro hilo. La espera va en un `while` para revisar la condición al despertar.

¿Por qué distinguir el valor del estado del recipiente?

Porque 0 también es un dato válido. Puedo dejar `amount` en cero al consumir, pero `producido` es lo que indica si hay un dato disponible.

c Preguntas de reflexión

¿Qué métodos de sincronización se deben utilizar (`wait()`, `notify()`, `synchronized`)?

`synchronized` permite que solo un hilo a la vez entre a los métodos del recipiente. `wait()` pausa el hilo y libera el bloqueo. `notify()` despierta a uno de los que esperan; en mi código uso `notifyAll()`, que los despierta a todos para que revisen su condición.

¿El programa lanza alguna excepción? ¿Por qué?

En la última prueba no hubo excepciones. `wait()` y `sleep()` pueden lanzar `InterruptedException` si el hilo es interrumpido. También puede aparecer `IllegalMonitorStateException` si uso `wait()` o `notify()` sin tener el bloqueo del objeto.

¿Es necesario modificar el encabezado de los métodos `produce` y `consume`?

Sí. Agregué `synchronized` y `throws InterruptedException` a ambos métodos. Como `wait()` puede lanzar esa excepción, las llamadas se manejan con `try-catch` en los hilos.

¿Qué se verificó al compilar y ejecutar nuevamente?

El programa compiló y, en la prueba, produjo y consumió del 0 al 9 en orden, sin pérdidas ni repeticiones. Ambos hilos terminaron. Falta contrastar los resultados con varias ejecuciones de la versión sin sincronización.

¿Qué ajustes quedan para mejorar la versión revisada?

Quitar el `if` que depende de `amount` y esperar directamente con `while (!producido)`. También escribir `amount = 0` para que se entienda mejor y terminar el hilo si ocurre una interrupción.